The medium

This is a small language for describing computations. We will build it around one game, rock, paper, scissors: choosing hands, playing rounds, keeping score, and asking a person to play.

Examples show how the pieces combine. Then a short shape collects the possibilities, with any restrictions beside them. When a shape changes, we show it again; the others stay as they were.

1. Things

First, we need the game's three hands:

```
Hand = rock | paper | scissors
```

This reads: a hand is rock, paper or scissors. `Hand` is a **type**: it tells us what kind of thing we mean. The names on the right are **constructors**. For now, each names a different thing.

A round also has an outcome:

```
Outcome = win | lose | draw
```

A hand is one of three choices. An outcome is one of three results. The names change; the shape stays the same:

```
T = c | …
```

`T` stands for a type's name, and `c` for a constructor. `| …` means we can list more constructors. These letters describe the shape; they aren't names in the game.

Each type is declared once, and each constructor belongs to one declaration. Types start with a capital letter; constructors don't.

A constructor can be written on its own, like `rock` or `win`. That is an **expression**, and it is already a **value**: nothing needs working out. With `e` for an expression and `v` for a value:

```
e ::= c
v ::= c
```

`::=` reads “has one of these shapes”. At this point, an expression can only be a constructor.

2. Functions

Paper beats rock, scissors beat paper, and rock beats scissors. We can put those three answers into one rule:

```
better : (Hand) → Hand
better = (rock) ↦ paper
       | (paper) ↦ scissors
       | (scissors) ↦ rock
```

`better` is a **function** with three **cases**, separated by `|`. Each case is what it is given, followed by what it gives back. `↦` reads “gives”.

The line above the definition gives its **function type**. It reads “better goes from a hand to a hand”. `→` describes the kinds of inputs and results; `↦` gives the answer in a particular case.

The input and result can have different types. This function gives the outcome of playing a hand against rock:

```
againstRock : (Hand) → Outcome
againstRock = (rock) ↦ draw
            | (paper) ↦ win
            | (scissors) ↦ lose
```

`better` takes a hand and gives a hand. `againstRock` takes a hand and gives an outcome.

To use a function, put an input after its name. We write `>>` for one step:

```
better(rock)
    >> paper
```

This is **applying** the function. We find the first case that matches the input, and replace the application with that case's answer. A constructor matches itself: rock matches rock.

`better(rock)` and `paper` are both expressions. Only `paper` is already a value: `better(rock)` still needs a step.

The input can itself be something to work out:

```
better(better(rock))
    >> better(paper)
    >> scissors
```

The inner application goes first. Once it gives us `paper`, the outer one can use it. This is how we combine functions: the result of one becomes the input of another. The type helps us see why this combination works: `better` gives back the same kind of thing it takes.

The two functions can combine too:

```
againstRock(better(rock))
    >> againstRock(paper)
    >> win
```

`better` gives back a hand, which is what `againstRock` takes. The result of the whole expression is an outcome.

We have added functions and application. `n` stands for a defined function's name, and `A` and `B` for declared types:

```
n : (A) → B              a function's type
n = (c) ↦ c | …          its cases

e ::= c                  a constructor
    | n(e)               an application
```

`A` and `B` may be the same or different types. Each `c` is a separate place: `(rock) ↦ paper` uses different constructors. A function has one or more cases, each taking one input. Each function name has one definition and is distinct from constructor names.

The `e` inside `n(e)` can be either form again: a constructor or another application. This allows nesting as deep as we need. For now, a function name appears only in an application, and each case gives a constructor directly. Values are still just constructors.

The nested steps have a shape too. If the selected cases give $n_2(c_1)$ `>>` c_2 and $n_1(c_2)$ `>>` c_3, then:

$$n_1(n_2(c_1)) \texttt{ >> } n_1(c_2) \texttt{ >> } c_3$$

The subscripts distinguish places, not necessarily different functions or constructors. `>>` describes a step; it isn't part of a program.

Naming an input and working out an answer

Suppose we want the hand that rock beats. That is scissors: the result of using `better` twice. The same works for every hand, so let's give it a name:

```
worse : (Hand) → Hand
worse = (a) ↦ better(better(a))
```

`a` is a **variable**: it wasn't declared as a constructor, so in a pattern it matches any value and names that value inside its case. Given `paper`, we put `paper` wherever this `a` occurs:

```
worse(paper)
    >> better(better(paper))
    >> better(scissors)
    >> rock
```

The part before `↦` is the **pattern**: it matches the input. The expression after `↦` is the **body**. The body waits until we apply the function, and only the case we use is worked out.

Naming an input doesn't make us use it. This function accepts a hand and always gives rock:

```
alwaysRock : (Hand) → Hand
alwaysRock = (a) ↦ rock

alwaysRock(paper)
    >> rock
```

We can also give the input back unchanged. This function changes rock to the hand that beats it, and keeps any other hand:

```
adjust : (Hand) → Hand
adjust = (rock) ↦ better(rock)
       | (a) ↦ a

adjust(rock)
    >> better(rock)
    >> paper
adjust(scissors)
    >> scissors
```

The first case has a constructor as its pattern and an application as its body. In the second, `a` names the input and the body gives it back. Both cases could match rock; trying them in order makes the second mean “otherwise”.

These examples cover the available kinds of case:

Pattern	Constructor as body	Variable as body	Application as body
A constructor	`(rock) ↦ paper`	No variable is named by this pattern	`(rock) ↦ better(rock)`
A variable	`(a) ↦ rock`	`(a) ↦ a`	`(a) ↦ better(better(a))`

An application's input can now be a constructor, a variable, or another application. We have seen all three: `better(rock)`, `better(a)` inside `worse`, and `better(better(a))`.

A case can now name its input and work out its answer. The definition, pattern and expression shapes change:

```
n = (p) ↦ e | …

p ::= c | x
e ::= c
    | x                  if this case's pattern is x
    | n(e)
```

`p` stands for a pattern and `x` for a variable. Every `e` place, whether a body or an input, can use any of the three forms. A variable must come from this case's pattern, even inside nested applications. A constructor pattern names no variable; outside a case there is none to use.

For now, a body can use function names already defined. A variable's name must differ from constructor names and defined function names. The rule for application is the same: work out the input, then use the first matching case. Its body may now need further steps, as `worse` and `adjust` show.

Matching can fail. `better(win)` gets **stuck**: `win` is a value, but none of the cases accepts it. The type line helps us see the mismatch. Writing a type doesn't change how the cases work.

Several inputs

To find who wins a round, we need two hands:

```
result : (Hand, Hand) → Outcome
result = (rock, scissors) ↦ win
       | (scissors, paper) ↦ win
       | (paper, rock) ↦ win
       | (rock, rock) ↦ draw
       | (paper, paper) ↦ draw
       | (scissors, scissors) ↦ draw
       | (a, b) ↦ lose
```

Each input matches its own pattern. The first three cases are wins for the first player; the next three are draws. The last case accepts anything left over, so it means “otherwise”.

```
result(paper, rock)       >>   win
result(rock, paper)       >>   lose
```

Several inputs are worked out from left to right:

```
result(better(rock), better(paper))
    >> result(paper, better(paper))
    >> result(paper, scissors)
    >> lose
```

A function can take any number of inputs, even none:

```
later : () → Hand
later = () ↦ better(rock)

later()
    >> better(rock)
    >> paper
```

With no inputs, we still have a body waiting to be used. Its application is what starts it.

Each pattern place can independently hold a constructor or a variable. A practice rule can use all four combinations for two inputs:

```
practice : (Hand, Hand) → Hand
practice = (rock, rock) ↦ paper
         | (rock, b) ↦ b
         | (a, rock) ↦ a
         | (a, b) ↦ better(a)
```

With two rocks, it chooses paper. With just one rock, it keeps the other hand. Otherwise it chooses the hand that beats the first. The cases are still tried in order.

Only the number of inputs changes:

```
n : (A, …) → B
n = (p, …) ↦ e | …

e ::= c
    | x                   if named by this case's patterns
    | n(e, …)
```

A comma followed by … means any number of these places, including none. Each input is an expression, so a call can mix constructors, available variables and other applications. Every case of one function has the same number of patterns, and the function takes that many inputs.

The variables in one case must have different names. In `(a, a)`, the same name would be given two inputs. To match two rocks, write `(rock, rock)`, as above.

3. Data and patterns

A round needs two hands. We can keep them together:

```
Round = round(Hand, Hand)
```

A `Round` is made with `round`, holding two hands:

```
round(paper, rock)
```

The first player chose paper, and the second chose rock. This is already a value. The constructor holds the two hands together. Values built with constructors are **data**.

To use those hands, write the same shape as a pattern:

```
judge : (Round) → Outcome
judge = (round(a, b)) ↦ result(a, b)

judge(round(paper, rock))
    >> result(paper, rock)
    >> win
```

`round(a, b)` takes the round apart: `a` names its first hand, and `b` its second. A body can put data together again:

```
flip : (Round) → Round
flip = (round(a, b)) ↦ round(b, a)

flip(round(paper, rock))
    >> round(rock, paper)
```

A constructor has one place for each thing it holds. A constructor that holds nothing, like `rock`, is written without parentheses.

Each content can itself be something to work out. We work out the contents from left to right before the data is a value:

```
round(better(rock), rock)
    >> round(paper, rock)
```

Each place in a pattern can be fixed or left open too. `round(rock, a)` fixes the first hand and leaves the second open. It matches `round(rock, paper)`, with `paper` as `a`, but doesn't match `round(paper, rock)`. `round(a, b)` leaves both hands open.

Data can hold data. A record can say that nothing was played, hold a round, or hold a round with its outcome:

```
Record = unplayed | played(Round) | scored(Round, Outcome)

unplayed
played(round(rock, paper))
scored(round(better(rock), rock), win)
    >> scored(round(paper, rock), win)
```

The constructors hold zero, one and two values. The pattern `scored(round(paper, a), outcome)` looks inside both layers of the last value: `a` names rock and `outcome` names win. A variable can also name a whole layer: `scored(thisRound, outcome)` keeps the round together.

Constructors can now hold contents. That changes declarations, expressions, values and patterns:

```
T = c(A, …) | …

e ::= c(e, …)
    | x                   if named by this case's patterns
    | n(e, …)
v ::= c(v, …)
p ::= c(p, …) | x
```

`A` and `B` still stand for declared types. For now, a declaration uses types declared before it. A constructor takes as many contents as its declaration lists; with none, write its name alone.

An `e` place accepts any expression; a `v` place needs a value. A `p` place accepts a constructor pattern or a variable, so patterns can nest as data does. All variables in one case's patterns must have different names; each is available throughout its body.

A type describes which contents fit. `round(win, lose)` can still be written: it is data, but it isn't a `Round`.

4. Functions as values

We used `better` twice to define `worse`. We can do the same with any rule from hands to hands:

```
twice : ((Hand) → Hand, Hand) → Hand
twice = (f, a) ↦ f(f(a))
```

Read its type from the outside: two inputs, then one result. The first input is a function from hands to hands. The second is a hand. The result is a hand.

```
twice(better, rock)
    >> better(better(rock))
    >> better(paper)
    >> scissors
```

`better` on its own is the function. `better(rock)` applies it. A function is a value, so we can pass it around before using it.

We can also write a function directly, without giving it a name:

```
(a) ↦ rock
```

This rule accepts an input and always gives back rock. An unnamed function can also have several cases, such as `(rock) ↦ paper | (a) ↦ a`. Named or unnamed, the same pattern and body rules apply.

We can pass the rule directly:

```
twice((a) ↦ rock, paper)
    >> ((a) ↦ rock)(((a) ↦ rock)(paper))
    >> ((a) ↦ rock)(rock)
    >> rock
```

Parentheses keep each function together. A body extends as far to the right as it can, so we group a function when something after it belongs outside its body.

Giving back a function

Suppose we want to choose the hand that a rule always gives back:

```
always : (Hand) → (Hand) → Hand
always = (a) ↦ (b) ↦ a
```

Given a hand, `always` makes a rule that ignores its own input and gives back that hand:

```
always(rock)
    >> (b) ↦ rock
```

The outer input went into the inner function's body, where `a` was. That body still waits until someone applies the new function:

```
always(rock)(paper)
    >> ((b) ↦ rock)(paper)
    >> rock
```

The type makes the two stages visible. `(Hand) → (Hand) → Hand` means “from a hand to a function from hands to hands”. What follows `→` stays together, just as a body does. Two stages, `f(a)(b)`, are different from giving one function two inputs, `f(a, b)`.

A function can take a rule and make another rule from it:

```
twofold : ((Hand) → Hand) → (Hand) → Hand
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(better)
    >> (a) ↦ better(better(a))
```

That is the rule used by `worse`. And the rule made by `always` can be passed to `twice`:

```
twice(always(rock), paper)
    >> … >> rock
```

`>>` shows one step. `>> … >>` abbreviates a sequence of steps to show where they lead.

Keeping names clear

A variable belongs to its case. Functions inside the body can use it, as `always` uses `a`. If an inner case introduces the same name, that inner name takes over there:

```
((a) ↦ (a) ↦ a)(rock)(paper)
    >> ((a) ↦ a)(paper)
    >> paper
```

The last `a` belongs to the inner case, so rock never replaces it. We replace a variable only where its name means that variable.

Variables cannot reuse a constructor's or a defined function's name. Each global name has one meaning, and each function is defined once. For now, a definition uses only functions defined before it; repetition will widen that rule.

Holding a function

A player can hold its rule:

```
Player = player((Hand) → Hand)

player(better)
player(always(rock))
```

The first player chooses the hand that beats the one it is given. The second always chooses rock. A pattern gets the rule out, and application uses it:

```
choose : (Player, Hand) → Hand
choose = (player(f), a) ↦ f(a)

choose(player(better), rock)
    >> better(rock)
    >> paper
```

A pattern can name a function, as `f` does here, but can't inspect its cases. We use a function by applying it.

An application has this shape:

```
e(e, …)
```

First work out the expression on the left to find the function. Then work out its inputs, from left to right, and apply it. In `always(rock)(paper)`, finding the function means working out `always(rock)` first.

If the value found isn't a function, or takes a different number of inputs, the application gets stuck:

```
twice(rock, paper)        gets stuck: rock isn't a function
twice(result, rock)       gets stuck: result takes two inputs
```

Each gets stuck when `f` is applied. The same rule applies to a name used directly: `better()` gets stuck because its input is missing.

What changed

Every value is now data or a function. We have seen both as inputs and results:

Input	Result	Example
Data	Data	`better(rock)`
Data	A function	`always(rock)`
A function, with data	Data	`twice(better, rock)`
A function	A function	`twofold(better)`

Data can hold either kind too: `round(rock, paper)` holds data, and `player(better)` holds a function. Three shapes change:

```
A ::= T
    | (A, …) → B

e ::= c(e, …)
    | x                   if named by a surrounding case
    | n
    | (p, …) ↦ e | …
    | e(e, …)

v ::= c(v, …)
    | n
    | (p, …) ↦ e | …
```

`A` and `B` now stand for either kind of type. The `e` to the left of an application's parentheses can be any expression too: it must produce a function when worked out. Every other `e` place has the same

choices.

A variable can come from this case or a surrounding case; the nearest one with that name wins. Patterns haven't changed: they take data apart or name a whole value, including a function.

5. Repeating

To keep score, we need numbers. We can make them from two constructors:

```
Nat = zero | succ(Nat)
```

`succ` means “one more than”. Zero is a number; one more than a number is another number. So `zero`, `succ(zero)` and `succ(succ(zero))` are zero, one and two.

The declaration uses its own name. A function can do that too:

```
add : (Nat, Nat) → Nat
add = (zero, b) ↦ b
    | (succ(a), b) ↦ succ(add(a, b))
```

Zero and `b` make `b`. One more than `a`, and `b`, make one more than `a` and `b`:

```
add(succ(succ(zero)), succ(zero))
    >> succ(add(succ(zero), succ(zero)))
    >> succ(succ(add(zero, succ(zero))))
    >> succ(succ(succ(zero)))
```

Each step takes a layer off the first input and puts it around the result. The same two cases work however many layers there are. This is **recursion**: using a name inside its own definition.

A win earns one point:

```
points : (Outcome) → Nat
points = (win) ↦ succ(zero)
       | (other) ↦ zero

points(draw)       >>   zero
```

Here the last case overlaps the first: it could match win too. Trying cases in order is what makes win worth a point and everything else worth zero.

Recursion needn't make progress:

```
loop : (Hand) → Hand
loop = (a) ↦ loop(a)

loop(rock)
    >> loop(rock), and so on forever
```

A function type doesn't promise to finish. `(Hand) → Hand` says that on hand inputs it doesn't get stuck, and any result is a hand. `loop` never finishes. The function `(rock) ↦ paper` doesn't fit: it gets stuck on paper or scissors.

Two functions can also refer to each other. These alternate between rock and paper as the number grows:

```
rockTurn : (Nat) → Hand
rockTurn = (zero) ↦ rock | (succ(a)) ↦ paperTurn(a)

paperTurn : (Nat) → Hand
paperTurn = (zero) ↦ paper | (succ(a)) ↦ rockTurn(a)

rockTurn(succ(succ(zero)))
    >> paperTurn(succ(zero))
    >> rockTurn(zero)
    >> rock
```

Types can refer to each other too:

```
Turn = turn(Hand, Reply) | finished
Reply = reply(Hand, Turn)

turn(rock, reply(paper, finished))
```

A turn holds a hand and a reply; the reply holds a hand and the next turn. Here the next turn is `finished`.

No expression, value or pattern shape changes. The name rule does: functions and types can now refer to themselves or to ones defined later. For now, a recursive type reference must go through data contents. Section 12 will allow function results too.

Working out an expression can finish with a value, get stuck, or go on forever.

6. Leaving a type open

A match is a list of rounds. The hands a player has shown are a list too. We can describe both with one declaration:

```
List(Item) = empty | cons(Item, List(Item))
```

`Item` leaves the type of each item open. A `List(Hand)` is empty, or a hand followed by a list of hands. A `List(Round)` has the same shape with rounds in those places.

```
cons(rock, cons(paper, empty))
```

This is a list of two hands. `cons` holds the first item and the remaining list. A function can follow that structure:

```
size : (List(Item)) → Nat
size = (empty) ↦ zero
     | (cons(a, rest)) ↦ succ(size(rest))

size(cons(rock, cons(paper, empty)))
    >> … >> succ(succ(zero))
```

In a signature, an undeclared capitalised name such as `Item`, written where a type goes, stands for an arbitrary type. Use the same choice everywhere that name occurs in that signature. The signature describes the function for every choice: `size` works for lists of hands, rounds, or any other kind of value. Types guide our reading; they aren't extra inputs to the function.

We can transform every item in a list:

```
each : ((From) → To, List(From)) → List(To)
each = (f, empty) ↦ empty
     | (f, cons(a, rest)) ↦ cons(f(a), each(f, rest))

each(better, cons(rock, cons(paper, empty)))
    >> … >> cons(paper, cons(scissors, empty))
each(points, cons(win, cons(draw, empty)))
    >> … >> cons(succ(zero), cons(zero, empty))
```

The first example changes hands into hands. The second changes outcomes into numbers. The signature shows the connection: the function takes the kind of item in the input list and gives the kind in the output list.

Two independent types can be left open too:

```
Pair(First, Second) = pair(First, Second)
```

`pair(win, zero)` is a `Pair(Outcome, Nat)`. The declaration's type variables belong only to that declaration. Their names must be different, and must not reuse a declared type's name. Types can nest: `List(List(Hand))` is a list of lists, and `List((Hand) → Hand)` is a list of rules.

Declarations gain type variables; types gain places to fill them:

```
T(X, …) = c(A, …) | …

A ::= T(A, …)
    | X                   if available in this declaration or signature
    | (A, …) → B
```

`X` stands for a type variable. In a declaration it must be listed on the left; in a signature it stands for an arbitrary type. Each type takes as many type arguments as its declaration lists. With none, write `Hand`, without parentheses.

Playing a match

We can now let two rules play. Each rule sees the other player's previous hand. `play` asks both for their next hands; `meet` puts those hands into a round and plays the remaining rounds:

```
play : (Nat, (Hand) → Hand, (Hand) → Hand, Hand, Hand) → List(Round)
play = (zero, f, g, a, b) ↦ empty
     | (succ(left), f, g, a, b) ↦ meet(left, f, g, f(b), g(a))

meet : (Nat, (Hand) → Hand, (Hand) → Hand, Hand, Hand) → List(Round)
meet = (left, f, g, a, b) ↦ cons(round(a, b), play(left, f, g, a, b))
```

The two definitions use each other. Two rounds of `better` against “always rock”, starting with rock as both previous hands:

```
play(succ(succ(zero)), better, always(rock), rock, rock)
    >> … >> cons(round(paper, rock), cons(round(paper, rock), empty))
```

And we can count the wins:

```
wins : (List(Round)) → Nat
wins = (empty) ↦ zero
     | (cons(this, rest)) ↦ add(points(judge(this)), wins(rest))

wins(play(succ(succ(zero)), better, always(rock), rock, rock))
    >> … >> succ(succ(zero))
```

7. The core together

Every value we have made is data or a function:

A value	How we make it	How we use it
Data	A constructor with its contents	A pattern that takes it apart
A function	Cases with patterns and bodies	Application to inputs

Contents, inputs and results can themselves be data or functions. A type describes which kind belongs in each place.

Here are all the shapes introduced so far:

```
T(X, …) = c(A, …) | …
n : (A, …) → B
n = (p, …) ↦ e | …
```

The complete expression shape is:

```
e ::= c(e, …)               data
    | x                     if named by a surrounding case
    | n                     a defined name
    | (p, …) ↦ e | …        a function
    | e(e, …)               an application

v ::= c(v, …)
    | n
    | (p, …) ↦ e | …
```

Each `e` place can hold any of the five forms. This gives a function returning a function, data holding an application, or an application finding its function through another application.

Patterns also fit inside patterns:

```
p ::= x                     name the whole value
    | c(p, …)               match its constructor and contents
```

For example, `cons(round(rock, a), rest)` looks inside both the list and its first round. It matches `cons(round(rock, paper), empty)`, with paper as `a` and the empty list as `rest`.

Types have three shapes:

```
A ::= T(A, …)               a declared type, filled with types
    | X                     a type variable available here
    | (A, …) → B            a function type
```

Each type place can hold any of these forms, as in `List((Hand) → Hand)`. In a declaration, `X` must be listed among its type variables. In a signature, it stands for an arbitrary type, chosen consistently.

The shapes say where pieces fit; matching and evaluation say what happens. A program gives declarations and definitions, then an expression to work out.

These pieces are enough to describe any computable transformation of data. Next we will let a computation ask for something while it runs: a hand someone chooses, for example.

8. Requests

Everything so far has been **saying**. Each step rewrites part of an expression into something that means the same: `worse(paper)` and `rock` are two ways of writing one hand. Computing can also be pictured another way, as steps that change a memory kept beside the program. That is **doing**. The two pictures compute the same things, but some ideas are natural only as doing: waiting for a person, keeping a score that changes, giving up partway.

The medium keeps saying underneath and brings doing back on top. It adds a new form only when an idea can't be said **locally** with the forms already there: either not at all, or only by changing parts of the program far from where the idea is used. Waiting for a person is the first such idea.

So far, we have chosen every hand in the program. To let a person choose, we need a way to ask:

```
Unit = unit

effect Console = read : () → Hand
               | show : (Outcome) → Unit
```

`Console` is an **effect**, declared by listing its **requests**. `read` takes no input and asks to be answered with a hand. `show` takes an outcome to display. Its answer is `unit`, the only value of `Unit`: there is nothing useful to give back, only something done.

An effect declaration has this shape:

```
effect F = r : (A, …) → B | …
```

`F` stands for an effect's name, and `r` for a request's name. Each request gives the types of its inputs and, after →, the type of its answer.

```
show(result(read(), rock))
```

This asks for a hand, finds its outcome against rock, and shows that outcome. The first thing it needs is `read()`. A request has no answer of its own, so the program waits there.

Suppose the person chooses paper. The answer goes where the request was:

```
show(result(paper, rock))
    >> show(win)
```

Now it waits at `show(win)`. Whatever runs the program displays win and answers `unit`. The whole expression then becomes that answer, and is finished.

A request can go inside data as well as inside an application:

```
round(read(), rock)
```

The round waits for the requested hand before it is a value.

Requests can take or return functions too. We can ask for a playing rule, or ask to save one:

```
effect Rules = askRule : () → ((Hand) → Hand)
             | saveRule : ((Hand) → Hand) → Unit

saveRule(better)
askRule()(rock)
saveRule(askRule())
```

The first gives a function to a request. The second asks for a function, then applies it to rock. The third uses the answer to one request as the input of another. The inputs and answer have the types in the declaration; they can be data or functions.

Expressions gain one alternative:

```
e ::= c(e, …)
    | x                   if named by a surrounding case
    | n
    | (p, …) ↦ e | …
    | e(e, …)
    | r(e, …)
```

A request can fill any e place. Its inputs are worked out from left to right, just like a function's. Each request belongs to one effect and takes its declared number of inputs. Values and patterns haven't changed: a request waits for an answer; it isn't a value.

Saying what a function may ask

We can give the game a name and keep it for later:

```
game : () → Unit ! {Console}
game = () ↦ show(result(read(), rock))
```

`! {Console}` reads “may ask Console”. Calling `game()` starts the requests. Until then, the body waits like any other function's body.

A person can supply a playing rule too:

```
human : (Hand) → Hand ! {Console}
human = (previous) ↦ read()
```

The rule ignores the previous hand and asks the person for the next one. Its result is a hand, provided the request is answered with a hand.

With no effects, leave out `! {…}`.

“May ask” includes not asking. So `better` also fits `(Hand) → Hand ! {Console}`. We can keep rules and people in one list:

```
cons(better, cons(human, empty))
```

Its type is `List((Hand) → Hand ! {Console})`. Several effects can be listed too: `() ↦ askRule()(read())` fits `() → Hand ! {Rules, Console}`. It asks for a rule, then a hand, and applies the rule to that hand.

The list describes requests left for something else to answer. If each receives an answer of its declared type, the function must avoid getting stuck and give the stated kind of result if it finishes.

Asking now or later

We can ask a person for a hand, then make a rule that always gives that hand:

```
pick : () → ((Hand) → Hand) ! {Console}
pick = () ↦ always(read())
```

Calling `pick()` asks once. If the person answers paper, it gives back `(b) ↦ paper`. Applying that rule asks nothing: the hand is already chosen.

Parentheses group a type just as they group an expression. Here they keep the returned rule's type, `((Hand) → Hand)`, together. The `! {Console}` outside those parentheses belongs to the call that makes the rule.

To return a rule that asks each time it is used:

```
askLater : () → ((Hand) → Hand ! {Console})
askLater = () ↦ human
```

Calling `askLater()` gives back `human` without asking. Calling the returned rule asks for a hand. The effect is inside the parentheses because it belongs to that rule.

Both calls may ask too: `() → ((Hand) → Hand ! {Console}) ! {Console}`. Each suffix belongs to the nearest preceding arrow in the same group. Without parentheses, arrows extend to the right.

Leaving the requests open

`each` applies a function to every item in a list. That function can be `human` too:

```
each(human, cons(rock, cons(paper, empty)))
```

This asks for two hands, from left to right, and collects the answers. If the person answers scissors and then rock, the result is `cons(scissors, cons(rock, empty))`.

The earlier signature described only functions that ask nothing. Replace it with this wider one; the definition stays the same:

```
each : ((From) → To ! {Asks}, List(From)) → List(To) ! {Asks}
```

`Asks` leaves a list of effects open, just as `From` and `To` leave types open. Choose the same list everywhere it occurs in this signature. For `human`, it is `Console`; for `better`, it can be empty. `each` may ask whatever the function it uses may ask.

An undeclared capitalised name in an effect list stands for a list, possibly empty. Known effects can go alongside it, as in `{Console, Asks}`. Order doesn't matter, and repeated effects count once. These are descriptions, not extra inputs to the function.

The type shape now includes possible effects. An entry in their list is a named effect or a list variable:

```
A ::= T(A, …)
    | X                   if available in this declaration or signature
    | (A, …) → B ! {E, …}

E ::= F                   a declared effect
    | S                   a list variable, only in a signature
```

`S` follows the rule just shown with `Asks`: one choice throughout its signature. One name cannot stand for both a type and an effect list.

9. Handlers

The program can answer a request itself. A **handler** goes around an expression and says how to answer:

```
handle better(read()) with
      (answer) ↦ answer
    | (read(), next) ↦ next(rock)
```

The first case receives the final value and gives it back. The second receives the request. It answers rock, then continues.

`next` is what remains to be done, written as a function. Here, what remains is to use `better` on the answer. It also keeps the same handler around that work:

```
(hand) ↦ handle better(hand) with
              (answer) ↦ answer
            | (read(), next) ↦ next(rock)
```

So `next(rock)` continues with rock, `better(rock)` becomes paper, and the handler's first case gives back paper. The whole handled expression comes to `paper`.

With a different answer, the same expression behaves differently. A handler using `next(paper)` would make `better(read())` come to scissors. The expression asks; the handler chooses how to answer.

Which handler answers?

A request goes to the nearest surrounding handler with a case for its name. A handler with no such case lets it pass:

```
handle (handle better(read()) with (hand) ↦ worse(hand)) with
      (answer) ↦ answer
    | (read(), next) ↦ next(rock)

    >> … >> rock
```

The inner handler has no case for `read`, so the outer one answers rock. Continuing still includes the inner handler: `better(rock)` gives paper, and the inner handler applies `worse` to paper, giving rock.

If no surrounding handler answers, the request reaches the outside, as it did when a person chose a hand. The outside answers once and the program continues.

A handler's case runs outside the handler it just used. Calling `next` puts that handler back around the remaining work. This matters if the case makes another request:

```
handle show(win) with
      (answer) ↦ answer
    | (show(outcome), next) ↦ next(show(outcome))
```

`outcome` names the request's input. The case sends that outcome to `show` outside this handler. Once the outside answers `unit`, `next` carries on and the handler returns `unit`. It doesn't repeatedly handle its

own request.

Having a case for a request's name is different from matching its inputs:

```
handle show(win) with
      (answer) ↦ answer
    | (show(lose), next) ↦ next(unit)
```

This gets stuck. The handler takes responsibility for `show`, but its only pattern doesn't match win. It doesn't pass the request on. Likewise, `handle paper with (rock) ↦ scissors` gets stuck because its final value matches no value case.

The shape with handlers

A handler is an expression too. Its result can fill an input, a data content or the place where an application finds its function. Here it returns a rule, which is then applied to paper:

```
(handle read() with
      (hand) ↦ always(hand)
    | (read(), next) ↦ next(rock))(paper)
    >> … >> rock
```

The handler supplies rock and makes the rule `always(rock)`. The final application uses that rule. The parentheses keep the handler together, so `(paper)` belongs to the application outside it.

Handlers may have value cases, request cases, or both. A value-only handler appeared in the nested example above. A request-only handler can stop at the request:

```
handle read() with (read(), next) ↦ rock
    >> rock
```

If a handled expression reaches a value and there is no matching value case, it gets stuck. As with functions, several cases of the same kind are tried in order.

Expressions gain a handler. Its cases have two forms, written `h`:

```
e ::= c(e, …)
    | x                   if named by a surrounding case
    | n
    | (p, …) ↦ e | …
    | e(e, …)
    | r(e, …)
    | handle e with h | …

h ::= (p) ↦ e              for a final value
    | (r(p, …), x) ↦ e     for a request; x names the rest
```

Each `e` place can hold any expression, including another handler. A request case has one pattern per input and one variable for the rest. All names introduced by a case are distinct and belong to its body; the nearest binding still wins. No other shape changes.

10. Choosing how to continue

Handlers are the last new expression form. The remaining examples combine the forms we have. Since `next` is a function, a case can use it once, several times, later, or never.

Stopping

A player can run out of prepared hands:

```
effect Forfeit = forfeit : () → Hand

firstOf : (List(Hand)) → Hand ! {Forfeit}
firstOf = (cons(a, rest)) ↦ a
        | (empty) ↦ forfeit()
```

The handler can make that a loss:

```
handle result(firstOf(empty), rock) with
      (outcome) ↦ outcome
    | (forfeit(), next) ↦ lose

    >> … >> lose
```

The case never applies `next`, so the round is never compared. The handler's answer becomes the result of the whole handled expression. An error that abandons the current computation works this way.

Trying several answers

Which hands would beat rock? We can answer `read()` three times and count the wins:

```
handle result(read(), rock) with
      (outcome) ↦ points(outcome)
    | (read(), next) ↦ add(next(rock), add(next(paper), next(scissors)))

    >> … >> succ(zero)
```

Each call to `next` starts from the same waiting point, with a different hand. Each result passes through the handler's first case, so each becomes a number: zero for rock, one for paper, zero for scissors.

Here `next` has type `(Hand) → Nat`. The request still needs a hand as its answer, but the handler turns the final outcome into a number. The type makes both ends visible.

This is search: try possible answers and combine what they lead to. We used ordinary functions and data to say what to do with the alternatives.

11. Keeping a score

A score changes as rounds are played. We can describe each change with a score before and a score after:

```
recordWith : (Outcome, Nat) → Pair(Outcome, Nat)
recordWith = (outcome, total) ↦ pair(outcome, add(points(outcome), total))

recordWith(win, zero)
    >> … >> pair(win, succ(zero))
recordWith(draw, succ(zero))
    >> … >> pair(draw, succ(zero))
```

To record two rounds, we take the score from the first result and pass it into the second call:

```
recordBothWith : (Round, Round, Nat) → Pair(List(Outcome), Nat)
recordBothWith = (first, second, total) ↦
    ((pair(a, afterFirst)) ↦
        ((pair(b, afterSecond)) ↦
            pair(cons(a, cons(b, empty)), afterSecond))
        (recordWith(judge(second), afterFirst)))
    (recordWith(judge(first), total))
```

Read from the first call: judge the first round and record it with `total`. Call its outcome `a` and its score `afterFirst`. Record the second round with that score, then give back both outcomes and the final score.

```
recordBothWith(round(paper, rock), round(rock, rock), zero)
    >> … >> pair(cons(win, cons(draw, empty)), succ(zero))
```

`recordBothWith` only passes the score along. A function that uses it would have to pass the score too. Every function between the place the score starts and the places it changes would need to carry it in and out. The core can say a changing score, but not locally: one small idea changes every function on the way. That is the kind of idea requests are for.

Requests let us mention the score where it is actually used:

```
effect State(Stored) = get : () → Stored
                     | put : (Stored) → Unit
```

An effect can leave types open, as a data type can. `State(Nat)` keeps a number; `State(Hand)` keeps a hand. `get()` asks for the current value. `put(a)` asks to replace it with `a` and is answered with `unit`.

Only effect declarations and effect-list entries change:

```
effect F(X, …) = r : (A, …) → B | …

E ::= F(A, …)
    | S                   a list variable, only in a signature
```

An effect's type variables follow the same rules as a data type's. Fill them with as many types as the declaration lists. With none, write `Console`, without parentheses.

In one effect list, each effect name has one choice of type arguments. Combining `State(Nat)` and `State(Hand)` isn't allowed: both use the same `get` and `put` names, so their requests couldn't be distinguished.

To do one thing, then return another:

```
then : (Unit, Item) → Item
then = (u, a) ↦ a
```

Inputs are worked out from left to right. The first input therefore happens before the second, even though the body uses only the second. We can record a win by getting the score, adding one, and putting it back:

```
record : (Outcome) → Outcome ! {State(Nat)}
record = (win) ↦ then(put(succ(get())), win)
       | (outcome) ↦ outcome
```

And record two rounds:

```
recordBoth : (Round, Round) → List(Outcome) ! {State(Nat)}
recordBoth = (first, second) ↦
    cons(record(judge(first)), cons(record(judge(second)), empty))
```

`recordBoth` passes no score. Its type says it may ask for state because `record` does. `then` itself is a pure function: the requests in its inputs happen before its body runs.

Answering the memory requests

A handler supplies the score. It passes the current score from one request to the next, but writes that passing in one place:

```
keep : (() → Result ! {State(Stored), Asks}, Stored) → Pair(Result, Stored) ! {Asks}
keep = (task, start) ↦
    (handle task() with
          (answer) ↦ ((current) ↦ pair(answer, current))
        | (get(), next) ↦ ((current) ↦ next(current)(current))
        | (put(value), next) ↦ ((current) ↦ next(unit)(value)))(start)
```

The signature says: give `keep` a task and an initial stored value; it gives back the task's result together with the final stored value. It answers the state requests itself. Other requests, described by `Asks`, pass through it.

The task is a function with no inputs. That keeps its work waiting until `task()` runs inside the handler. Given the work directly, we would run it before the handler was there.

Every case returns a function waiting for the current score:

- At a final value: pair that value with the score.
- At `get()`: answer with the score and continue with the same score.
- At `put(value)`: answer `unit` and continue with `value` as the score.

The `(start)` at the end supplies the first score; the parentheses around the handler keep it out of the last case's body. The two applications in `next(current)(current)` have different jobs. The first answers the request. The second supplies the score to the function returned by the continued handler.

```
keep(() ↦ recordBoth(round(paper, rock), round(rock, rock)), zero)
    >> … >> pair(cons(win, cons(draw, empty)), succ(zero))
```

The first round wins and increases the score. The second draws and leaves it alone. The task gives back the outcomes; `keep` adds the final score.

Order now matters in a visible way:

```
keep(() ↦ then(put(succ(zero)), get()), zero)
    >> … >> pair(succ(zero), succ(zero))

keep(() ↦ ((saved) ↦ then(put(succ(zero)), saved))(get()), zero)
    >> … >> pair(zero, succ(zero))
```

In the first, we put one and then get one. In the second, we get zero first, call it `saved`, then put one. The final score is the same, but the task returns the value it read earlier.

The handler never inspects the stored value. The same definition keeps a hand:

```
keep(() ↦ then(put(paper), get()), rock)
    >> … >> pair(paper, paper)
```

The task can also ask to show its outcome:

```
keep(() ↦ then(show(record(win)), get()), zero)
```

`keep` answers the score requests. `show(win)` passes through it to the outside. Once that is answered with `unit`, the expression comes to `pair(succ(zero), succ(zero))`. Here `Stored` is `Nat`, `Result` is `Nat`, and `Asks` is `Console`.

Different effects can appear in the same expression. Each request still finds the nearest handler with a case for its name:

```
handle (handle then(show(win), get()) with
              (answer) ↦ answer
            | (show(outcome), next) ↦ next(unit)) with
      (answer) ↦ answer
    | (get(), next) ↦ next(zero)

    >> … >> zero
```

The inner handler answers `show`. The request for `get` passes through it to the outer handler. The result is the number supplied there.

Two scores, two places

One `keep` supplies one store. Two nested handlers for the same `get` and `put` names don't give us two stores we can choose between: the nearer handler answers both names. To keep a score for each player, give each score a **place**, and include that place in each request:

```
Place = mine | yours

effect Scores = look : (Place) → Nat
              | change : (Place, Nat) → Unit
```

`look(mine)` asks for my score; `change(yours, zero)` asks to set yours to zero. Adding a point works for either place:

```
bump : (Place) → Unit ! {Scores}
bump = (place) ↦ change(place, succ(look(place)))
```

The handler keeps both scores. Each case returns a function waiting for the two current values, `a` and `b`:

```
keepScores : (() → Result ! {Scores, Asks}, Nat, Nat) → Result ! {Asks}
keepScores = (task, first, second) ↦
    (handle task() with
          (answer) ↦ ((a, b) ↦ answer)
        | (look(mine), next) ↦ ((a, b) ↦ next(a)(a, b))
        | (look(yours), next) ↦ ((a, b) ↦ next(b)(a, b))
        | (change(mine, value), next) ↦ ((a, b) ↦ next(unit)(value, b))
        | (change(yours, value), next) ↦ ((a, b) ↦ next(unit)(a, value)))(first, second)
```

Looking answers with the selected score and keeps both scores as they were. Changing answers `unit` and replaces only the selected score. At a final value, the handler gives back the task's answer.

```
keepScores(() ↦ then(bump(mine), pair(look(mine), look(yours))), zero, zero)
    >> … >> pair(succ(zero), zero)
```

A place is a value, so we can pass it to a function or hold it in data. Two names can refer to the same place:

```
keepScores(() ↦ ((samePlace) ↦ then(bump(samePlace), look(mine)))(mine), zero, zero)
    >> … >> succ(zero)
```

`samePlace` was given `mine`. Adding through `samePlace` changes what looking through `mine` finds: both refer to the same score. The scores live in the handler; a place tells it which score we mean.

This uses the same requests, patterns, functions and data as before. Here the two places are fixed. More places can be represented with numbers and their stored values kept in a list.

12. Continuing later

A handler can return the rest of the work instead of carrying it out immediately. Then whoever receives it decides when to continue.

Let a player give its hands one at a time:

```
effect Give = give : (Hand) → Unit

cycle : (Hand) → Unit ! {Give}
cycle = (a) ↦ then(give(a), cycle(better(a)))
```

Starting with rock, it gives rock, paper, scissors, then rock again. It keeps asking forever.

A handler can stop at each hand and return that hand with the rest:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves : (() → Unit ! {Give}) → (Unit) → Moves
moves = (task) ↦ (u) ↦
    handle task() with
          (answer) ↦ done
        | (give(a), next) ↦ gave(a, next)
```

`moves` gives back a function waiting for `unit`. When applied, it starts the task. At `give(a)`, it returns `a` together with `next`, without applying `next`.

The returned `next` also waits for `unit`, the answer to `give`. Applying it reaches the next hand. In these lines, `next`, `after` and `again` name the functions returned alongside the hands:

```
moves(() ↦ cycle(rock))(unit)       >> … >>   gave(rock, next)
next(unit)                         >> … >>   gave(paper, after)
after(unit)                        >> … >>   gave(scissors, again)
```

The player is infinite, but each request for a move reaches one hand. This is a generator: its caller asks for each next item when it needs it.

`Moves` also shows a recursive type containing a function. A `gave` holds a hand and a function that can produce more `Moves`. Such a sequence can keep unfolding indefinitely; it needn't eventually reach `done`.

The type forms haven't changed, but the rule for recursive references widens. In section 5, a type referred back to itself through data contents. It may now do so through a function's result too, as in `(Unit) → Moves`.

This language excludes recursive references through a function's input: `Test = test((Test) → Outcome)` is not an allowed declaration. The restriction applies through other type declarations and type arguments too. An answer to a request is an input supplied from outside, so a recursive reference may not pass through a request's answer type either.

Taking turns

Two generators can play each other. `turns` asks each for a hand, and `joinMoves` builds a round if both supplied one:

```
turns : (Nat, (Unit) → Moves, (Unit) → Moves) → List(Round)
turns = (zero, first, second) ↦ empty
      | (succ(left), first, second) ↦ joinMoves(left, first(unit), second(unit))

joinMoves : (Nat, Moves, Moves) → List(Round)
joinMoves = (left, gave(a, first), gave(b, second)) ↦
                cons(round(a, b), turns(left, first, second))
          | (left, first, second) ↦ empty
```

The new functions use no new forms. `first` and `second` hold what remains of each player, and `turns` chooses when to continue each one.

```
rocks : () → Unit ! {Give}
rocks = () ↦ then(give(rock), rocks())

turns(succ(succ(succ(zero))), moves(() ↦ cycle(rock)), moves(rocks))
    >> … >> cons(round(rock, rock),
                 cons(round(paper, rock),
                      cons(round(scissors, rock), empty)))
```

Each player describes its own moves. `turns` controls their order. This gives a small example of cooperative scheduling: tasks give control back at requests, and a scheduler decides which task continues.

13. The whole medium

Here are the pieces together. The letters stand for parts of the language:

Letter	What it stands for
`e`, `v`	An expression, a value
`p`, `h`	A pattern, a handler case
`x`, `n`	A variable, a defined function's name
`c`, `r`	A constructor, a request
`T`, `F`	A declared type's name, an effect's name
`A`, `B`, `X`	Types, and a variable for a type
`E`, `S`	An entry in an effect list, and a variable for a list of effects

`e, …` means any number of expressions, including none; likewise for the other letters. An item followed by `| …` allows one or more alternatives. Omit parentheses for a type or effect with no type arguments, and for a constructor with no contents. `>>` shows one evaluation step; `>> … >>` abbreviates several. Neither is part of a program.

Declarations and definitions

```
T(X, …) = c(A, …) | …                 a type and its constructors
effect F(X, …) = r : (A, …) → B | …   an effect and its requests

n : A                                 a function's type, when written
n = (p, …) ↦ e | …                    its definition
```

A program gives declarations and definitions, then an expression to work out. A signature can describe a particular use of a more general function. It doesn't change evaluation or enforce its description: checking whether a function fits is outside the medium.

Expressions, values and patterns

```
e ::= c(e, …)                         data
    | x                               if named by a surrounding case
    | n                               a defined function
    | (p, …) ↦ e | …                  a function written out
    | e(e, …)                         an application
    | r(e, …)                         a request
    | handle e with h | …             a handled expression

v ::= c(v, …)                         data whose contents are values
    | n                               a defined function
    | (p, …) ↦ e | …                  a function written out

p ::= x                               match any value and name it
    | c(p, …)                         match a constructor and its contents

h ::= (p) ↦ e                         for a final value
    | (r(p, …), x) ↦ e                for a request, naming the rest as x
```

Every `e` place accepts any expression form, subject to the rules below. The shapes can nest without a fixed limit.

Types

```
A ::= T(A, …)                        a declared type, filled with types
    | X                              a type variable available here
    | (A, …) → B ! {E, …}            a function type

E ::= F(A, …)                        a declared effect, filled with types
    | S                              a list variable, only in a signature
```

Parentheses can group any type: `(A)` means the same type as `A`. Leave out `! {…}` when there are no effects. Each effect suffix belongs to the nearest preceding arrow in the same group. Without grouping, the result after an arrow extends to the right.

A declaration can use declared types and effects, and its own type variables. Those variables have distinct names and cannot reuse declared type or effect names. A type or effect takes as many type arguments as its declaration lists.

In a signature, an undeclared capitalised name stands for an arbitrary type or, as an effect-list entry, an arbitrary list of effects. One name cannot stand for both. Each choice stays the same throughout that signature.

An effect-list variable can stand for no effects, one, or several. Combine them with the named effects. Order doesn't matter; repetitions count once. Each effect name must have the same type arguments throughout the combined list. List variables occur only in signatures.

A value belongs to a declared type when it is made with one of that type's constructors and its contents belong to the types declared for them. Fill the declaration's type variables with the types supplied to it.

A function belongs to a function type if it takes the stated number of inputs and meets three conditions. Give it inputs of the stated types, and answer its requests with values of their declared answer types. However those answers are chosen:

- It never gets stuck.
- Each request it leaves unanswered belongs to a listed effect and has inputs of the declared types.
- Any final result belongs to `B`.

It may also run forever. Fill in an effect's type arguments when reading its requests: under `State(Nat)`, `put` takes a `Nat` and `get` is answered with one.

A function fits a signature with variables when it belongs to the type made by every allowed choice of them, for types and effect lists alike.

Recursive types include all values consistent with these descriptions, including values that keep unfolding through functions. A recursive path may pass through data contents or function results, but must never put the recursive type in a function input or a request-answer type. This also applies when the path goes through a type argument in another declaration.

Names and grouping

Each global name has one declaration or definition, and a defined function has at most one signature. Each constructor belongs to one type declaration, and each request to one effect declaration. Types

and effects, including their type variables, start with capitals; names used in expressions start without capitals.

A definition can use any defined name, including itself or a name defined later. A type declaration can refer to itself or other declared types, subject to the recursive-type restriction above.

In a pattern, a declared constructor means that constructor. Any other name is a variable. The variables of one case are distinct, including the variable for the rest in a request case. They cannot reuse a constructor, request or defined function's name.

A variable is in scope in its case's body, including functions inside it. The nearest case that binds a name determines what the name means. A name in an expression with no such binding must be declared or defined.

A constructor or request uses its declared number of contents or inputs, also in patterns. All cases of a function have the same number of patterns. Applying it to a different number of inputs gets stuck.

Parentheses group an expression or a type. Function and handler bodies extend to the right; group them when following text belongs outside the body.

Working out an expression

In an application, first work out what is being applied, then its inputs from left to right. In data or a request, work out the contents or inputs from left to right. In a handler, work on the expression it surrounds. Nothing happens inside a function until it is applied.

Applying a function to values takes one step. Find the first case whose patterns all match. The application becomes that case's body, with each matched value put wherever its variable means it. A defined function uses the cases of its definition in the same way.

Matching follows the shape of data. A constructor pattern needs the same constructor, with each content matching its pattern. A variable matches the whole value at that place. If no case matches, applying gets stuck. Applying data also gets stuck.

When a handled expression becomes a value, the handler takes one step to the body of its first matching value case. Put the matched values in place in that body. With no matching value case, it gets stuck.

When the next work is a request with value inputs, find the nearest surrounding handler with a case for that request's name. The handler takes one step to the body of its first matching request case. Put the inputs' matched values in place, and give the extra variable the rest of the work. If the handler names the request but none of its patterns matches, it gets stuck.

The rest is a function taking one answer. Its body is the interrupted expression with that answer where the request was, surrounded by the same handler. Intervening handlers that let the request pass are kept too. Use a fresh name for the answer so it cannot interfere with any existing variable. The request case itself runs outside the handler just used.

With no handler for a request, evaluation waits for the outside. Its answer replaces the request, and evaluation continues. A whole computation therefore has four possibilities: it returns a value, gets stuck, waits for an answer, or continues forever.

The shape to remember

Piece	How it is made	How it is used
Data	A constructor holds values	A pattern takes them apart
A function	Cases connect patterns to bodies	Application supplies inputs
A request	An operation asks with inputs	A handler supplies a response and controls continuation

Names let functions refer to themselves and to one another. Types make contents, inputs and results visible. The larger examples combine these pieces: state passes a value between requests; a generator keeps the rest for later.

Why these pieces

Functions alone could already compute anything; even hands and numbers can be written as functions. The core gives data its own form anyway, so that it can be seen.

Requests and handlers let us ask for something where it is needed, without passing it through every function on the way. Errors, search, state, generators and taking turns use those forms; they need no further ones.

Types describe what kinds of things we mean. They don't change what a program does.

So when you meet something new, whether a feature of another language or a precise idea of your own, ask what data, what functions and what requests it involves, what kinds of things they are, and how they are combined.